第9章 接 口

接口和内部类为我们提供了一种将接口与实现分离的更加结构化的方法。

这种机制在编程语言中并不通用。例如，C++对这些概念只有间接的支持。在Java中存在语言关键字这个事实表明人们认为这些思想是很重要的，以至于要提供对它们的直接支持。

首先，我们将学习抽象类，它是普通的类与接口之间的一种中庸之道。尽管在构建具有某些未实现方法的类时，你的第一想法可能是创建接口，但是抽象类仍旧是用于此目的的一种重要而必须的工具。因为你不可能总是使用纯接口。

9.1 抽象类和抽象方法

在第8章所有“乐器”的例子中，基类**Instrument**中的方法往往是“哑”（dummy）方法。若要调用这些方法，就会出现一些错误。这是因为**Instrument**类的目的是为它的所有导出类创建一个*通用接口*。

在那些示例中，建立这个通用接口的唯一理由是，不同的子类可以用不同的方式表示此接口。通用接口建立起一种基本形式，以此表示所有导出类的共同部分。另一种说法是将**Instrument**类称作*抽象基类*，或简称*抽象类*。

如果我们只有一个像**Instrument**这样的抽象类，那么该类的对象几乎没有任何意义。我们创建抽象类是希望通过这个通用接口操纵一系列类。因此，**Instrument**只是表示了一个接口，没有具体的实现内容；因此，创建一个**Instrument**对象没有什么意义，并且我们可能还想阻止使用者这样做。通过让**Instrument**中的所有方都产生错误，就可以实现这个目的。但是这样做会将错误信息延迟到运行时才获得，并且需要在客户端进行可靠、详尽的测试。所以最好是在编译时捕获这些问题。

为此，Java提供一个叫做*抽象方法*[⊖]的机制，这种方法是不完整的；仅有声明而没有方法体。下面是抽象方法声明所采用的语法：

```
abstract void f();
```

包含抽象方法的类叫做*抽象类*。如果一个类包含一个或多个抽象方法，该类必须被限定为**抽象的**。（否则，编译器就会报错。）

如果一个抽象类不完整，那么当我们试图产生该类的对象时，编译器会怎样处理呢？由于为抽象类创建对象是不安全的，所以我们会从编译器那里得到一条出错消息。这样，编译器会确保抽象类的纯粹性，我们不必担心会误用它。

如果从一个抽象类继承，并想创建该新类的对象，那么就必须为基类中的所有抽象方法提供方法定义。如果不这样做（可以选择不做），那么导出类便也是抽象类，且编译器将会强制我们用**abstract**关键字来限定这个类。

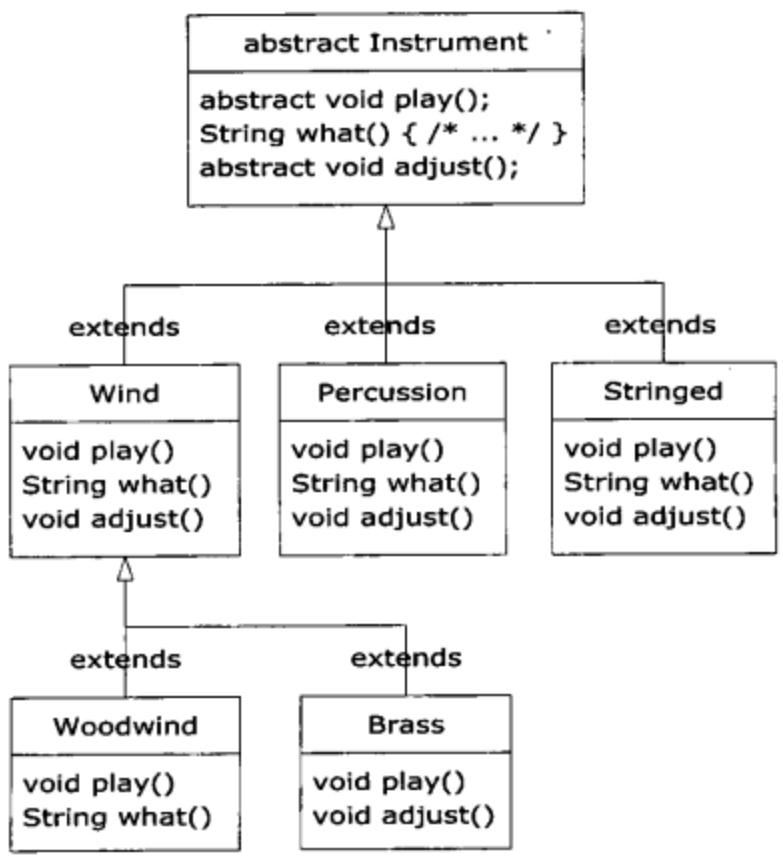
我们也可能会创建一个没有任何抽象方法的抽象类。考虑这种情况：如果有一个类，让其

⊖ 对于C++程序设计员来说，这相当于C++语言中的*纯虚函数*。

包含任何**abstract**方法都显得没有实际意义，而且我们也想要阻止产生这个类的任何对象，那么这时这样做就很有用了。

第8章**Instrument**类可以很容易地转化成**abstract**类。既然使某个类成为抽象类并不需要所有的方法都是抽象的，所以仅需将某些方法声明为抽象的即可。如下图所示：

312



下面是修改过的“管弦乐器”的例子，其中采用了抽象类和抽象方法：

```
//: interfaces/music4/Music4.java
// Abstract classes and methods.
package interfaces.music4;
import polymorphism.music.Note;
import static net.mindview.util.Print.*;

abstract class Instrument {
  private int i; // Storage allocated for each
  public abstract void play(Note n);
  public String what() { return "Instrument"; }
  public abstract void adjust();
}

class Wind extends Instrument {
  public void play(Note n) {
    print("Wind.play() " + n);
  }
  public String what() { return "Wind"; }
  public void adjust() {}
}

class Percussion extends Instrument {
  public void play(Note n) {
    print("Percussion.play() " + n);
  }
  public String what() { return "Percussion"; }
  public void adjust() {}
}

class Stringed extends Instrument {
  public void play(Note n) {
    print("Stringed.play() " + n);
```

313

```
  }
  public String what() { return "Stringed"; }
  public void adjust() {}
}

class Brass extends Wind {
  public void play(Note n) {
    print("Brass.play() " + n);
  }
  public void adjust() { print("Brass.adjust()"); }
}

class Woodwind extends Wind {
  public void play(Note n) {
    print("Woodwind.play() " + n);
  }
  public String what() { return "Woodwind"; }
}

public class Music4 {
  // Doesn't care about type, so new types
  // added to the system still work right:
  static void tune(Instrument i) {
    // ...
    i.play(Note.MIDDLE_C);
  }
  static void tuneAll(Instrument[] e) {
    for(Instrument i : e)
      tune(i);
  }
  public static void main(String[] args) {
    // Upcasting during addition to the array:
    Instrument[] orchestra = {
      new Wind(),
      new Percussion(),
      new Stringed(),
      new Brass(),
      new Woodwind()
    };
    tuneAll(orchestra);
  }
} /* Output:
Wind.play() MIDDLE_C
Percussion.play() MIDDLE_C
Stringed.play() MIDDLE_C
Brass.play() MIDDLE_C
Woodwind.play() MIDDLE_C
*///:~
```

314

我们可以看出，除了基类，实际上并没有什么改变。

创建抽象类和抽象方法非常有用，因为它们可以使类的抽象性明确起来，并告诉用户和编译器打算怎样来使用它们。抽象类还是很有用的重构工具，因为它们使得我们可以很容易地将公共方法沿着继承层次结构向上移动。

练习1：(1) 修改第8章练习9中的**Rodent**，使其成为一个抽象类。只要有可能，就将**Rodent**的方法声明为抽象方法。

练习2：(1) 创建一个不包含任何抽象方法的抽象类，并验证我们不能为该类创建任何实例。

练习3：(2) 创建一个基类，让它包含抽象方法**print()**，并在导出类中覆盖该方法。覆盖后的方法版本可以打印导出类中定义的某个整型变量的值。在定义该变量之处，赋予它非零值。在基类的构造器中调用这个方法。现在，在**main()**方法中，创建一个导出类对象，然后调用它的**print()**方法。请解释发生的情形。

练习4：(3) 创建一个不包含任何方法的抽象类，从它那里导出一个类，并添加一个方法。创建一个静态方法，它可以接受指向基类的引用，将其向下转型到导出类，然后再调用该静态方法。在**main()**中，展现它的运行情况。然后，为基类中的方法加上**abstract**声明，这样就不再需要进行向下转型。

315

9.2 接口

interface关键字使抽象的概念更向前迈进了一步。**abstract**关键字允许人们在类中创建一个或多个没有任何定义的方法——提供了接口部分，但是没有提供任何相应的具体实现，这些实现是由此类的继承者创建的。**interface**这个关键字产生一个完全抽象的类，它根本就没有提供任何具体实现。它允许创建者确定方法名、参数列表和返回类型，但是没有任何方法体。接口只提供了形式，而未提供任何具体实现。

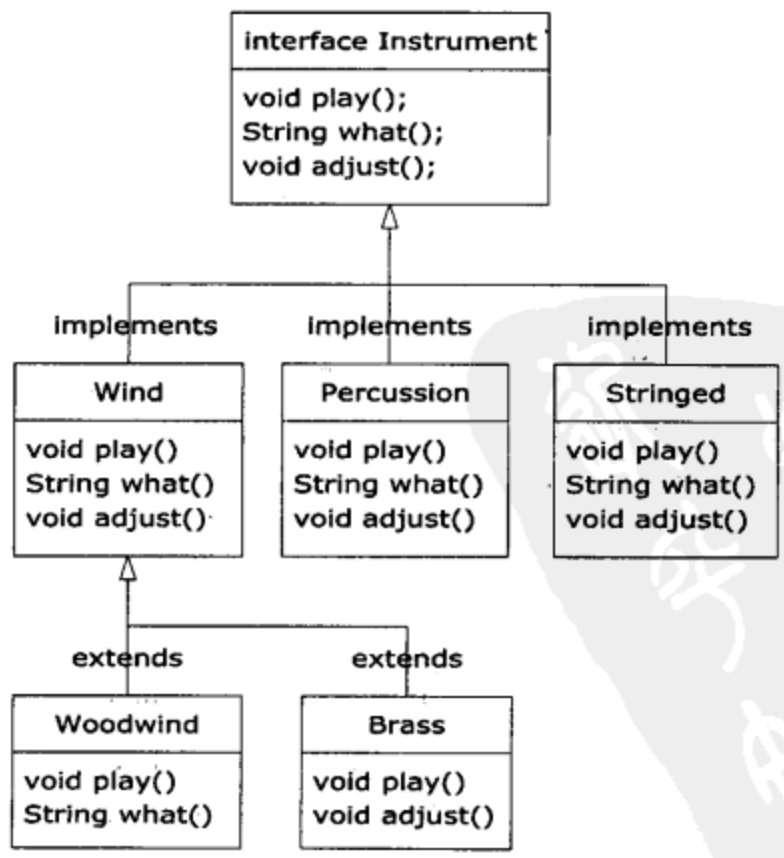
一个接口表示："所有实现了该特定接口的类看起来都像这样"。因此，任何使用某特定接口的代码都知道可以调用该接口的哪些方法，而且仅需知道这些。因此，接口被用来建立类与类之间的协议。(某些面向对象编程语言使用关键字protocol来完成这一功能。)

但是，**interface**不仅仅是一个极度抽象的类，因为它允许人们通过创建一个能够被向上转型为多种基类的类型，来实现某种类似多重继承的特性。

要想创建一个接口，需要用**interface**关键字来替代**class**关键字。就像类一样，可以在**interface**关键字前面添加**public**关键字（但仅限于该接口在与其同名的文件中被定义）。如果不添加**public**关键字，则它只具有包访问权限，这样它就只能在同一个包内可用。接口也可以包含域，但是这些域隐式地是**static**和**final**的。

要让一个类遵循某个特定接口（或者是一组接口），需要使用**implements**关键字，它表示："**interface**只是它的外貌，但是现在我要声明它是如何工作的。"除此之外，它看起来还很像继承。"乐器"示例的图说明了这一点：

316



可以从**Woodwind**和**Brass**类中看到，一旦实现了某个接口，其实现就变成了一个普通的类，就可以按常规方式扩展它。

可以选择在接口中显式地将方法声明为**public**的，但即使你不这么做，它们也是**public**的。因此，当要实现一个接口时，在接口中被定义的方法必须被定义为是**public**的；否则，它们将只能得到默认的包访问权限，这样在方法被继承的过程中，其可访问权限就被降低了，这是Java编译器所不允许的。

读者可以在修改过的**Instrument**的例子中看到这一点。要注意的是，在接口中的每一个方法确实都只是一个声明，这是编译器所允许的在接口中唯一能够存在的事物。此外，在**Instrument**中没有任何方法被声明为是**public**的，但是它们自动就都是**public**的：

317

```
//: interfaces/music5/Music5.java
// Interfaces.
package interfaces.music5;
import polymorphism.music.Note;
import static net.mindview.util.Print.*;

interface Instrument {
  // Compile-time constant:
  int VALUE = 5; // static & final
  // Cannot have method definitions:
  void play(Note n); // Automatically public
  void adjust();
}

class Wind implements Instrument {
  public void play(Note n) {
    print(this + ".play() " + n);
  }
  public String toString() { return "Wind"; }
  public void adjust() { print(this + ".adjust()"); }
}

class Percussion implements Instrument {
  public void play(Note n) {
    print(this + ".play() " + n);
  }
  public String toString() { return "Percussion"; }
  public void adjust() { print(this + ".adjust()"); }
}

class Stringed implements Instrument {
  public void play(Note n) {
    print(this + ".play() " + n);
  }
  public String toString() { return "Stringed"; }
  public void adjust() { print(this + ".adjust()"); }
}

class Brass extends Wind {
  public String toString() { return "Brass"; }
}

class Woodwind extends Wind {
  public String toString() { return "Woodwind"; }
}
public class Music5 {
  // Doesn't care about type, so new types
  // added to the system still work right:
  static void tune(Instrument i) {
    // ...
    i.play(Note.MIDDLE_C);
  }
  static void tuneAll(Instrument[] e) {
    for(Instrument i : e)
      tune(i);
```

318

```
    }
    public static void main(String[] args) {
      // Upcasting during addition to the array:
      Instrument[] orchestra = {
        new Wind(),
        new Percussion(),
        new Stringed(),
        new Brass(),
        new Woodwind()
      };
      tuneAll(orchestra);
    }
  } /* Output:
  Wind.play() MIDDLE_C
  Percussion.play() MIDDLE_C
  Stringed.play() MIDDLE_C
  Brass.play() MIDDLE_C
  Woodwind.play() MIDDLE_C
  *///:~
```

此实例的这个版本还有另外一处改动：**what()**方法已经被修改为**toString()**方法，因为**toString()**的逻辑正是**what()**要实现的逻辑。由于**toString()**方法是根类**Object**的一部分，因此它不需要出现在接口中。

余下的代码其工作方式都是相同的。无论是将其向上转型为称为**Instrument**的普通类，还是称为**Instrument**的抽象类，或是称为**Instrument**的接口，都不会有问题。它的行为都是相同的。事实上，你可以在**tune()**方法中看到，没有任何依据来证明**Instrument**是一个普通类、抽象类，还是一个接口。

319

练习5：(2) 在某个包内创建一个接口，内含三个方法，然后在另一个包中实现此接口。

练习6：(2) 证明接口内所有的方法都自动是**public**的。

练习7：(1) 修改第8章中的练习9，使**Rodent**成为一个接口。

练习8：(2) 在**polymorphism.Sandwich.java**中，创建接口**FastFood**并添加合适的方法，然后修改**Sandwich**以实现**FastFood**接口。

练习9：(3) 重构**Music5.java**，将在**Wind**、**Percussion**和**Stringed**中的公共方法移入一个抽象类中。

练习10：(3) 修改**Music5.java**，添加**Playable**接口。将**play()**的声明从**Instrument**中移到**Playable**中。通过将**Playable**包括在**implements**列表中，把**Playable**添加到导出类中。修改**tune()**，使它接受**Playable**而不是**Instrument**作为参数。

9.3 完全解耦

只要一个方法操作的是类而非接口，那么你就只能使用这个类及其子类。如果你想要将这个方法应用于不在此继承结构中的某个类，那么你就会触霉头了。接口可以在很大程度上放宽这种限制，因此，它使得我们可以编写可复用性更好的代码。

例如，假设有一个**Processor**类，它有一个**name()**方法；另外还有一个**process()**方法，该方法接受输入参数，修改它的值，然后产生输出。这个类做为基类而被扩展，用来创建各种不同类型的**Processor**。在本例中，**Processor**的子类将修改**String**对象（注意，返回类型可以是协变类型，而非参数类型）：

```
//: interfaces/classprocessor/Apply.java
package interfaces.classprocessor;
import java.util.*;
import static net.mindview.util.Print.*;
```

```
class Processor {
  public String name() {
    return getClass().getSimpleName();
  }
  Object process(Object input) { return input; }
}

class Upcase extends Processor {
  String process(Object input) { // Covariant return
    return ((String)input).toUpperCase();
  }
}

class Downcase extends Processor {
  String process(Object input) {
    return ((String)input).toLowerCase();
  }
}

class Splitter extends Processor {
  String process(Object input) {
    // The split() argument divides a String into pieces:
    return Arrays.toString(((String)input).split(" "));
  }
}

public class Apply {
  public static void process(Processor p, Object s) {
    print("Using Processor " + p.name());
    print(p.process(s));
  }
  public static String s =
    "Disagreement with beliefs is by definition incorrect";
  public static void main(String[] args) {
    process(new Upcase(), s);
    process(new Downcase(), s);
    process(new Splitter(), s);
  }
} /* Output:
Using Processor Upcase
DISAGREEMENT WITH BELIEFS IS BY DEFINITION INCORRECT
Using Processor Downcase
disagreement with beliefs is by definition incorrect
Using Processor Splitter
[Disagreement, with, beliefs, is, by, definition,
incorrect]
*///:~
```

320

Apply.process()方法可以接受任何类型的**Processor**，并将其应用到一个**Object**对象上，然后打印结果。像本例这样，创建一个能够根据所传递的参数对象的不同而具有不同行为的方法，被称为策略设计模式。这类方法包含所要执行的算法中固定不变的部分，而“策略”包含变化的部分。策略就是传递进去的参数对象，它包含要执行的代码。这里，**Processor**对象就是一个策略，在**main()**中可以看到有三种不同类型的策略应用到了**String**类型的**s**对象上。

321

split()方法是**String**类的一部分，它接受**String**类型的对象，并以传递进来的参数作为边界，将该**String**对象分隔开，然后返回一个数组**String[]**。它在这里被用来当作创建**String**数组的快捷方式。

现在假设我们发现了一组电子滤波器，它们看起来好像适用于**Apply.process()**方法：

```
//: interfaces/filters/Waveform.java
package interfaces.filters;

public class Waveform {
```

```
  private static long counter;
  private final long id = counter++;
  public String toString() { return "Waveform " + id; }
} ///:~
```

```
//: interfaces/filters/Filter.java
package interfaces.filters;

public class Filter {
  public String name() {
    return getClass().getSimpleName();
  }
  public Waveform process(Waveform input) { return input; }
} ///:~
```

```
//: interfaces/filters/LowPass.java
package interfaces.filters;

public class LowPass extends Filter {
  double cutoff;
  public LowPass(double cutoff) { this.cutoff = cutoff; }
  public Waveform process(Waveform input) {
    return input; // Dummy processing
  }
} ///:~
```

322

```
//: interfaces/filters/HighPass.java
package interfaces.filters;

public class HighPass extends Filter {
  double cutoff;
  public HighPass(double cutoff) { this.cutoff = cutoff; }
  public Waveform process(Waveform input) { return input; }
} ///:~
```

```
//: interfaces/filters/BandPass.java
package interfaces.filters;

public class BandPass extends Filter {
  double lowCutoff, highCutoff;
  public BandPass(double lowCut, double highCut) {
    lowCutoff = lowCut;
    highCutoff = highCut;
  }
  public Waveform process(Waveform input) { return input; }
} ///:~
```

Filter与**Processor**具有相同的接口元素，但是因为它并非继承自**Processor**——因为**Filter**类的创建者压根不清楚你想要将它用作**Processor**——因此你不能将**Filter**用于**Apply.process()**方法，即便这样做可以正常运行。这里主要是因为**Apply.process()**方法和**Processor**之间的耦合过紧，已经超出了所需要的程度，这就使得应该复用**Apply.process()**的代码时，复用却被禁止了。另外还需要注意的是它们的输入和输出都是**Waveform**。

但是，如果**Processor**是一个接口，那么这些限制就会变得松动，使得你可以复用结构该接口的**Apply.process()**。下面是**Processor**和**Apply**的修改版本：

```
//: interfaces/interfaceprocessor/Processor.java
package interfaces.interfaceprocessor;

public interface Processor {
  String name();
  Object process(Object input);
} ///:~
```

```
//: interfaces/interfaceprocessor/Apply.java
package interfaces.interfaceprocessor;
import static net.mindview.util.Print.*;
```

323

```
public class Apply {
  public static void process(Processor p, Object s) {
    print("Using Processor " + p.name());
    print(p.process(s));
  }
} ///:~
```

复用代码的第一种方式是客户端程序员遵循该接口来编写他们自己的类，就像下面这样：

```
//: interfaces/interfaceprocessor/StringProcessor.java
package interfaces.interfaceprocessor;
import java.util.*;

public abstract class StringProcessor implements Processor{
  public String name() {
    return getClass().getSimpleName();
  }
  public abstract String process(Object input);
  public static String s =
    "If she weighs the same as a duck, she's made of wood";
  public static void main(String[] args) {
    Apply.process(new Upcase(), s);
    Apply.process(new Downcase(), s);
    Apply.process(new Splitter(), s);
  }
}

class Upcase extends StringProcessor {
  public String process(Object input) { // Covariant return
    return ((String)input).toUpperCase();
  }
}

class Downcase extends StringProcessor {
  public String process(Object input) {
    return ((String)input).toLowerCase();
  }
}

class Splitter extends StringProcessor {
  public String process(Object input) {
    return Arrays.toString(((String)input).split(" "));
  }
} /* Output:
Using Processor Upcase
IF SHE WEIGHS THE SAME AS A DUCK, SHE'S MADE OF WOOD
Using Processor Downcase
if she weighs the same as a duck, she's made of wood
Using Processor Splitter
[If, she, weighs, the, same, as, a, duck,, she's, made, of,
wood]
*///:~
```

324

但是，你经常碰到的情况是你无法修改你想要使用的类。例如，在电子滤波器的例子中，类库是被发现而非被创建的。在这些情况下，可以使用*适配器*设计模式。适配器中的代码将接受你所拥有的接口，并产生你所需要的接口，就像下面这样：

```
//: interfaces/interfaceprocessor/FilterProcessor.java
package interfaces.interfaceprocessor;
import interfaces.filters.*;

class FilterAdapter implements Processor {
  Filter filter;
  public FilterAdapter(Filter filter) {
    this.filter = filter;
  }
```

```
  public String name() { return filter.name(); }
  public Waveform process(Object input) {
    return filter.process((Waveform)input);
  }
}

public class FilterProcessor {
  public static void main(String[] args) {
    Waveform w = new Waveform();
    Apply.process(new FilterAdapter(new LowPass(1.0)), w);
    Apply.process(new FilterAdapter(new HighPass(2.0)), w);
    Apply.process(
      new FilterAdapter(new BandPass(3.0, 4.0)), w);
  }
} /* Output:
Using Processor LowPass
Waveform 0
Using Processor HighPass
Waveform 0
Using Processor BandPass
Waveform 0
*///:~
```

325

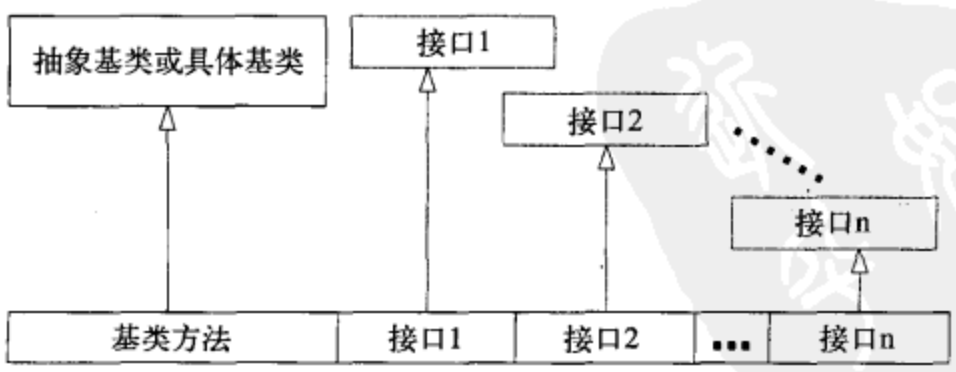
在这种使用适配器的方式中，**FilterAdapter**的构造器接受你所拥有的接口**Filter**，然后生成具有你所需要的**Processor**接口的对象。你可能还注意到了，在**FilterAdapter**类中用到了代理。

将接口从具体实现中解耦使得接口可以应用于多种不同的具体实现，因此代码也就更具可复用性。

练习11：(4) 创建一个类，它有一个方法用于接受一个**String**类型的参数，生成的结果是将该参数中每一对字符进行互换。对该类进行适配，使得它可以用于**interfaceprocessor.Apply.process()**。

9.4 Java中的多重继承

接口不仅仅只是一种更纯粹形式的抽象类，它的目标比这要高。因为接口是根本没有任何具体实现的——也就是说，没有任何与接口相关的存储；因此，也就无法阻止多个接口的组合。这一点是很有价值的，因为你有时需要去表示“一个**x**是一个**a**和一个**b**以及一个**c**”。在C++中，组合多个类的接口的行为被称作*多重继承*。它可能会使你背负很沉重的包袱，因为每个类都有一个具体实现。在Java中，你可以执行相同的行为，但是只有一个类可以有具体实现；因此，通过组合多个接口，C++中的问题是不会在Java中发生的：



326

在导出类中，不强制要求必须有一个是抽象的或“具体的”（没有任何抽象方法的）基类。如果要从一个非接口的类继承，那么只能从一个类去继承。其余的基元素都必须是接口。需要将所有的接口名都置于**implements**关键字之后，用逗号将它们一一隔开。可以继承任意多个接口，并可以向上转型为每个接口，因为每一个接口都是一个独立类型。下面的例子展示了一个具体类组合数个接口之后产生了一个新类：

```
//: interfaces/Adventure.java
// Multiple interfaces.

interface CanFight {
  void fight();
}

interface CanSwim {
  void swim();
}

interface CanFly {
  void fly();
}

class ActionCharacter {
  public void fight() {}
}

class Hero extends ActionCharacter
    implements CanFight, CanSwim, CanFly {
  public void swim() {}
  public void fly() {}
}

public class Adventure {
  public static void t(CanFight x) { x.fight(); }
  public static void u(CanSwim x) { x.swim(); }
  public static void v(CanFly x) { x.fly(); }
  public static void w(ActionCharacter x) { x.fight(); }
  public static void main(String[] args) {
    Hero h = new Hero();
    t(h); // Treat it as a CanFight
    u(h); // Treat it as a CanSwim
    v(h); // Treat it as a CanFly
    w(h); // Treat it as an ActionCharacter
  }
} ///:~
```

327

可以看到，**Hero**组合了具体类**ActionCharacter**和接口**CanFight**、**CanSwim**和**CanFly**。当通过这种方式将一个具体类和多个接口组合到一起时，这个具体类必须放在前面，后面跟着的才是接口（否则编译器会报错）。

注意，**CanFight**接口与**ActionCharacter**类中的**fight()**方法的特征签名是一样的，而且，在**Hero**中并没有提供**fight()**的定义。可以扩展接口，但是得到的只是另一个接口。当想要创建对象时，所有的定义首先必须都存在。即使**Hero**没有显式地提供**fight()**的定义，其定义也因**ActionCharacter**而随之而来，这样就使得创建**Hero**对象成为了可能。

在**Adventure**类中，可以看到有四个方法把上述各种接口和具体类作为参数。当**Hero**对象被创建时，它可以被传递给这些方法中的任何一个，这意味着它依次被向上转型为每一个接口。由于Java中这种设计接口的方式，使得这项工作并不需要程序员付出任何特别的努力。

一定要记住，前面的例子所展示的就是使用接口的核心原因：为了能够向上转型为多个基类型（以及由此而带来的灵活性）。然而，使用接口的第二个原因却是与使用抽象基类相同：防止客户端程序员创建该类的对象，并确保这仅仅是建立一个接口。这就带来了一个问题：我们应该使用接口还是抽象类？如果要创建不带任何方法定义和成员变量的基类，那么就应该选择接口而不是抽象类。事实上，如果知道某事物应该成为一个基类，那么第一选择应该是使它成为一个接口（该主题在本章的总结中将再次讨论）。

练习12：(2) 在**Adventure.java**中，按照其他接口的样式，增加一个**CanClimb**接口。

练习13：(2) 创建一个接口，并从该接口继承两个接口，然后从后面两个接口多重继承第三个接口[⊖]。

328

9.5 通过继承来扩展接口

通过继承，可以很容易地在接口中添加新的方法声明，还可以通过继承在新接口中组合数个接口。这两种情况都可以获得新的接口，就像在下例中所看到的：

```
//: interfaces/HorrorShow.java
// Extending an interface with inheritance.

interface Monster {
  void menace();
}

interface DangerousMonster extends Monster {
  void destroy();
}

interface Lethal {
  void kill();
}

class DragonZilla implements DangerousMonster {
  public void menace() {}
  public void destroy() {}
}

interface Vampire extends DangerousMonster, Lethal {
  void drinkBlood();
}

class VeryBadVampire implements Vampire {
  public void menace() {}
  public void destroy() {}
  public void kill() {}
  public void drinkBlood() {}
}

public class HorrorShow {
  static void u(Monster b) { b.menace(); }
  static void v(DangerousMonster d) {
    d.menace();
    d.destroy();
  }
  static void w(Lethal l) { l.kill(); }
  public static void main(String[] args) {
    DangerousMonster barney = new DragonZilla();
    u(barney);
    v(barney);
    Vampire vlad = new VeryBadVampire();
    u(vlad);
    v(vlad);
    w(vlad);
  }
} ///:~
```

329

DangerousMonster是**Monster**的直接扩展，它产生了一个新接口。**DragonZilla**中实现了这个接口。

在**Vampire**中使用的语法仅适用于接口继承。一般情况下，只可以将**extends**用于单一类，

⊖ 这可以说明接口是如何防止在C++多重继承中所产生的“菱形问题”的。

但是可以引用多个基类接口。就像所看到的，只需用逗号将接口名一一分隔开即可。

练习14：(2) 创建三个接口，每个接口都包含两个方法。继承出一个接口，它组合了这三个接口并添加了一个新方法。创建一个实现了该新接口并且继承了某个具体类的类。现在编写四个方法，每一个都接受这四个接口之一作为参数。在**main()**方法中，创建这个类的对象，并将其传递给这四个方法。

练习15：(2) 将前一个练习修改为：创建一个抽象类，并将其继承到一个导出类中。

9.5.1 组合接口时的名字冲突

在实现多重继承时，可能会碰到一个小陷阱。在前面的例子中，**CanFight**和**ActionCharacter**都有一个相同的**void fight()**方法。这不是问题所在，因为该方法在二者中是相同的。相同的方法不会有什么问题，但是如果它们的签名或返回类型不同，又会怎么样呢？这有一个例子：

```
//: interfaces/InterfaceCollision.java
package interfaces;

interface I1 { void f(); }
interface I2 { int f(int i); }
interface I3 { int f(); }
class C { public int f() { return 1; } }

class C2 implements I1, I2 {
  public void f() {}
  public int f(int i) { return 1; } // overloaded
}

class C3 extends C implements I2 {
  public int f(int i) { return 1; } // overloaded
}

class C4 extends C implements I3 {
  // Identical, no problem:
  public int f() { return 1; }
}

// Methods differ only by return type:
//! class C5 extends C implements I1 {}
//! interface I4 extends I1, I3 {} ///:~
```

330

此时困难来了，因为覆盖、实现和重载令人不快地搅在了一起，而且重载方法仅通过返回类型是区分不开的。当撤销最后两行的注释时，下列错误消息就说明了这一切：

InterfaceCollision.java:23: f( ) in C cannot implement f( ) in I1; attempting to use incompatible return type
found : int
required: void
InterfaceCollision.java:24: Interfaces I3 and I1 are incompatible; both define f( ), but with different return type

在打算组合的不同接口中使用相同的方法名通常会造成代码可读性的混乱，请尽量避免这种情况。

9.6 适配接口

接口最吸引人的原因之一就是允许同一个接口具有多个不同的具体实现。在简单的情况中，它的体现形式通常是一个接受接口类型的方法，而该接口的实现和向该方法传递的对象则取决

331

于方法的使用者。

因此，接口的一种常见用法就是前面提到的策略设计模式，此时你编写一个执行某些操作的方法，而该方法将接受一个同样是你指定的接口。你主要就是要声明："你可以用任何你想要的对象来调用我的方法，只要你的对象遵循我的接口。"这使得你的方法更加灵活、通用，并更具可复用性。

例如，Java SE5的**Scanner**类（在第13章中就更多地了解它）的构造器接受的就是一个**Readable**接口。你会发现**Readable**没有用作Java标准类库中其他任何方法的参数，它是单独为**Scanner**创建的，以使得**Scanner**不必将其参数限制为某个特定类。通过这种方式，**Scanner**可以作用于更多的类型。如果你创建了一个新的类，并且想让**Scanner**可以作用于它，那么你就应该让它成为**Readable**，就像下面这样：

```
//: interfaces/RandomWords.java
// Implementing an interface to conform to a method.
import java.nio.*;
import java.util.*;

public class RandomWords implements Readable {
  private static Random rand = new Random(47);
  private static final char[] capitals =
    "ABCDEFGHIJKLMNOPQRSTUVWXYZ".toCharArray();
  private static final char[] lowers =
    "abcdefghijklmnopqrstuvwxyz".toCharArray();
  private static final char[] vowels =
    "aeiou".toCharArray();
  private int count;
  public RandomWords(int count) { this.count = count; }
  public int read(CharBuffer cb) {
    if(count-- == 0)
      return -1; // Indicates end of input
    cb.append(capitals[rand.nextInt(capitals.length)]);
    for(int i = 0; i < 4; i++) {
      cb.append(vowels[rand.nextInt(vowels.length)]);
      cb.append(lowers[rand.nextInt(lowers.length)]);
    }
    cb.append(" ");
    return 10; // Number of characters appended
  }
  public static void main(String[] args) {
    Scanner s = new Scanner(new RandomWords(10));
    while(s.hasNext())
      System.out.println(s.next());
  }
} /* Output:
Yazeruyac
Fowenucor
Goeazimom
Raeuuacio
Nuoadesiw
Hageaikux
Ruqicibui
Numasetih
Kuuuuozog
Waqizeyoy
*///:~
```

332

Readable接口只要求实现**read()**方法，在**read()**内部，将输入内容添加到**CharBuffer**参数中（有多种方法可以实现此目的，请查看**CharBuffer**的文档），或者在没有任何输入时返回−1。

假设你有一个还未实现**Readable**的类，怎样才能让**Scanner**作用于它呢？下面这个类就是一个例子，它可以产生随机浮点数：

```
//: interfaces/RandomDoubles.java
import java.util.*;

public class RandomDoubles {
  private static Random rand = new Random(47);
  public double next() { return rand.nextDouble(); }
  public static void main(String[] args) {
    RandomDoubles rd = new RandomDoubles();
    for(int i = 0; i < 7; i ++)
      System.out.print(rd.next() + " ");
  }
} /* Output:
0.7271157860730044 0.5309454508634242 0.16020656493302599
0.18847866977771732 0.5166020801268457 0.2678662084200585
0.2613610344283964
*///:~
```

333

我们再次使用了适配器模式，但是在本例中，被适配的类可以通过继承和实现**Readable**接口来创建。因此，通过使用**interface**关键字提供的伪多重继承机制，我们可以生成既是**RandomDoubles**又是**Readable**的新类：

```
//: interfaces/AdaptedRandomDoubles.java
// Creating an adapter with inheritance.
import java.nio.*;
import java.util.*;

public class AdaptedRandomDoubles extends RandomDoubles
implements Readable {
  private int count;
  public AdaptedRandomDoubles(int count) {
    this.count = count;
  }
  public int read(CharBuffer cb) {
    if(count-- == 0)
      return -1;
    String result = Double.toString(next()) + " ";
    cb.append(result);
    return result.length();
  }
  public static void main(String[] args) {
    Scanner s = new Scanner(new AdaptedRandomDoubles(7));
    while(s.hasNextDouble())
      System.out.print(s.nextDouble() + " ");
  }
} /* Output:
0.7271157860730044 0.5309454508634242 0.16020656493302599
0.18847866977771732 0.5166020801268457 0.2678662084200585
0.2613610344283964
*///:~
```

因为在这种方式中，我们可以在任何现有类之上添加新的接口，所以这意味着让方法接受接口类型，是一种让任何类都可以对该方法进行适配的方式。这就是使用接口而不是类的强大之处。

练习16：(3) 创建一个类，它将生成一个**char**序列，适配这个类，使其可以成为**Scanner**对象的一种输入。

334

9.7 接口中的域

因为你放入接口中的任何域都自动是**static**和**final**的，所以接口就成为了一种很便捷的用来创建常量组的工具。在Java SE5之前，这是产生与C或C++中的**enum**（枚举类型）具有相同效果的类型的唯一途径。因此在Java SE5之前的代码中你会看到下面这样的代码：

```
//: interfaces/Months.java
// Using interfaces to create groups of constants.
package interfaces;

public interface Months {
  int
    JANUARY = 1, FEBRUARY = 2, MARCH = 3,
    APRIL = 4, MAY = 5, JUNE = 6, JULY = 7,
    AUGUST = 8, SEPTEMBER = 9, OCTOBER = 10,
    NOVEMBER = 11, DECEMBER = 12;
} ///:~
```

请注意，Java中标识具有常量初始化值的**static final**时，会使用大写字母的风格（在一个标识符中用下划线来分隔多个单词）。接口中的域自动是**public**的，所以没有显式地指明这一点。

有了Java SE5，你就可以使用更加强大而灵活的**enum**关键字，因此，使用接口来群组常量已经显得没什么意义了。但是，当你阅读遗留的代码时，在许多情况下你可能还是会碰到这种旧的习惯用法（www.MindView.net上关于本书的补充材料中，包含有关在Java SE5之前使用接口来生成枚举类型的方式的完整描述）。在第19章中可以看到更多的关于使用**enum**的细节说明。

练习17：(2) 证明在接口中的域隐式地是**static**和**final**的。

9.7.1 初始化接口中的域

在接口中定义的域不能是“空**final**”，但是可以被非常量表达式初始化。例如：

335

```
//: interfaces/RandVals.java
// Initializing interface fields with
// non-constant initializers.
import java.util.*;

public interface RandVals {
  Random RAND = new Random(47);
  int RANDOM_INT = RAND.nextInt(10);
  long RANDOM_LONG = RAND.nextLong() * 10;
  float RANDOM_FLOAT = RAND.nextLong() * 10;
  double RANDOM_DOUBLE = RAND.nextDouble() * 10;
} ///:~
```

既然域是**static**的，它们就可以在类第一次被加载时初始化，这发生在任何域首次被访问时。这里给出了一个简单的测试：

```
//: interfaces/TestRandVals.java
import static net.mindview.util.Print.*;

public class TestRandVals {
  public static void main(String[] args) {
    print(RandVals.RANDOM_INT);
    print(RandVals.RANDOM_LONG);
    print(RandVals.RANDOM_FLOAT);
    print(RandVals.RANDOM_DOUBLE);
  }
} /* Output:
8
-32032247016559954
-8.5939291E18
5.779976127815049
*///:~
```

当然，这些域不是接口的一部分，它们的值被存储在该接口的静态存储区域内。

9.8 嵌套接口

接口可以嵌套在类或其他接口中[⊖]。这揭示了许多非常有趣的特性：

```
//: interfaces/nesting/NestingInterfaces.java
package interfaces.nesting;
class A {
  interface B {
    void f();
  }
  public class BImp implements B {
    public void f() {}
  }
  private class BImp2 implements B {
    public void f() {}
  }
  public interface C {
    void f();
  }
  class CImp implements C {
    public void f() {}
  }
  private class CImp2 implements C {
    public void f() {}
  }
  private interface D {
    void f();
  }
  private class DImp implements D {
    public void f() {}
  }
  public class DImp2 implements D {
    public void f() {}
  }
  public D getD() { return new DImp2(); }
  private D dRef;
  public void receiveD(D d) {
    dRef = d;
    dRef.f();
  }
}

interface E {
  interface G {
    void f();
  }
  // Redundant "public":
  public interface H {
    void f();
  }
  void g();
  // Cannot be private within an interface:
  //! private interface I {}
}

public class NestingInterfaces {
  public class BImp implements A.B {
    public void f() {}
  }
  class CImp implements A.C {
    public void f() {}
  }
  // Cannot implement a private interface except
```

336

337

⊖ 感谢Martin Danner在研讨会上提出这个问题。

```
    // within that interface's defining class:
    //! class DImp implements A.D {
    //!  public void f() {}
    //! }
    class EImp implements E {
      public void g() {}
    }
    class EGImp implements E.G {
      public void f() {}
    }
    class EImp2 implements E {
      public void g() {}
      class EG implements E.G {
        public void f() {}
      }
    }
    public static void main(String[] args) {
      A a = new A();
      // Can't access A.D:
      //! A.D ad = a.getD();
      // Doesn't return anything but A.D:
      //! A.DImp2 di2 = a.getD();
      // Cannot access a member of the interface:
      //! a.getD().f();
      // Only another A can do anything with getD():
      A a2 = new A();
      a2.receiveD(a.getD());
    }
  } ///:~
```

338

在类中嵌套接口的语法是相当显而易见的，就像非嵌套接口一样，可以拥有**public**和“包访问”两种可视性。

作为一种新添加的方式，接口也可以被实现为**private**的，就像在**A.D**中所看到的（相同的语法既适用于嵌套接口，也适用于嵌套类）。那么**private**的嵌套接口能带来什么好处呢？读者可能会猜想，它只能够被实现为**DImp**中的一个**private**内部类，但是**A.DImp2**展示了它同样可以被实现为**public**类。但是，**A.DImp2**只能被其自身所使用。你无法说它实现了一个**private**接口**D**。因此，实现一个**private**接口只是一种方式，它可以强制该接口中的方法定义不要添加任何类型信息（也就是说，不允许向上转型）。

getD()方法使我们陷入了一个进退两难的境地，这个问题与**private**接口相关：它是一个返回对**private**接口的引用的**public**方法。你对这个方法的返回值能做些什么呢？在**main()**中，可以看到数次尝试使用返回值的行为都失败了。只有一种方式可成功，那就是将返回值交给有权使用它的对象。在本例中，是另一个**A**通过**receiveD()**方法来实现的。

接口**E**说明接口彼此之间也可以嵌套。然而，作用于接口的各种规则，特别是所有的接口元素都必须是**public**的，在此都会被严格执行。因此，嵌套在另一个接口中的接口自动就是**public**的，而不能声明为**private**的。

NestingInterfaces展示了嵌套接口的各种实现方式。特别要注意的是，当实现某个接口时，并不需要实现嵌套在其内部的任何接口。而且，**private**接口不能在定义它的类之外被实现。

添加这些特性的最初原因可能是出于对严格的语法一致性的考虑，但是我总认为，一旦你了解了某种特性，就总能够找到它的用武之地。

9.9 接口与工厂

接口是实现多重继承的途径，而生成遵循某个接口的对象的典型方式就是工厂方法设计模

式。这与直接调用构造器不同，我们在工厂对象上调用的是创建方法，而该工厂对象将生成接口的某个实现的对象。理论上，通过这种方式，我们的代码将完全与接口的实现分离，这就使得我们可以透明地将某个实现替换为另一个实现。下面的实例展示了工厂方法的结构：

339

```
//: interfaces/Factories.java
import static net.mindview.util.Print.*;

interface Service {
  void method1();
  void method2();
}

interface ServiceFactory {
  Service getService();
}

class Implementation1 implements Service {
  Implementation1() {} // Package access
  public void method1() {print("Implementation1 method1");}
  public void method2() {print("Implementation1 method2");}
}

class Implementation1Factory implements ServiceFactory {
  public Service getService() {
    return new Implementation1();
  }
}

class Implementation2 implements Service {
  Implementation2() {} // Package access
  public void method1() {print("Implementation2 method1");}
  public void method2() {print("Implementation2 method2");}
}

class Implementation2Factory implements ServiceFactory {
  public Service getService() {
    return new Implementation2();
  }
}

public class Factories {
  public static void serviceConsumer(ServiceFactory fact) {
    Service s = fact.getService();
    s.method1();
    s.method2();
  }
  public static void main(String[] args) {
    serviceConsumer(new Implementation1Factory());
    // Implementations are completely interchangeable:
    serviceConsumer(new Implementation2Factory());
  }
} /* Output:
Implementation1 method1
Implementation1 method2
Implementation2 method1
Implementation2 method2
*///:~
```

340

如果不是用工厂方法，你的代码就必须在某处指定将要创建的**Service**的确切类型，以便调用合适的构造器。

为什么我们想要添加这种额外级别的间接性呢？一个常见的原因是想要创建框架：假设你正在创建一个对弈游戏系统，例如，在相同的棋盘上下国际象棋和西洋跳棋：

```
//: interfaces/Games.java
```

```
// A Game framework using Factory Methods.
import static net.mindview.util.Print.*;

interface Game { boolean move(); }
interface GameFactory { Game getGame(); }

class Checkers implements Game {
  private int moves = 0;
  private static final int MOVES = 3;
  public boolean move() {
    print("Checkers move " + moves);
    return ++moves != MOVES;
  }
}

class CheckersFactory implements GameFactory {
  public Game getGame() { return new Checkers(); }
}

class Chess implements Game {
  private int moves = 0;
  private static final int MOVES = 4;
  public boolean move() {
    print("Chess move " + moves);
    return ++moves != MOVES;
  }
}

class ChessFactory implements GameFactory {
  public Game getGame() { return new Chess(); }
}

public class Games {
  public static void playGame(GameFactory factory) {
    Game s = factory.getGame();
    while(s.move())
      ;
  }
  public static void main(String[] args) {
    playGame(new CheckersFactory());
    playGame(new ChessFactory());
  }
} /* Output:
Checkers move 0
Checkers move 1
Checkers move 2
Chess move 0
Chess move 1
Chess move 2
Chess move 3
*///:~
```

341

如果**Games**类表示一段复杂的代码，那么这种方式就允许你在不同类型的游戏中复用这段代码。你可以再想象一些能够从这个模式中受益的更加精巧的游戏。

在下一章中，你将可以看到另一种更加优雅的工厂实现方式，那就是使用匿名内部类。

练习18：(2) 创建一个**Cycle**接口及其**Unicycle**、**Bicycle**和**Tricycle**实现。对每种类型的**Cycle**都创建相应的工厂，然后编写代码使用这些工厂。

342

练习19：(3) 使用工厂方法来创建一个框架，它可以执行抛硬币和掷骰子功能。

9.10 总结

“确定接口是理想选择，因而应该总是选择接口而不是具体的类。”这其实是一种引诱。当

然，对于创建类，几乎在任何时刻，都可以替代为创建一个接口和一个工厂。

许多人都掉进了这种诱惑的陷阱，只要有可能就去创建接口和工厂。这种逻辑看起来好像是因为需要使用不同的具体实现，因此总是应该添加这种抽象性。这实际上已经变成了一种草率的设计优化。

任何抽象性都应该是应真正的需求而产生的。当必需时，你应该重构接口而不是到处添加额外级别的间接性，并由此带来的额外的复杂性。这种额外的复杂性非常显著，如果你让某人去处理这种复杂性，只是因为你意识到由于以防万一而添加了新接口，而没有其他更有说服力的原因，那么好吧，如果我碰上了这种事，那么就会质疑此人所作的所有设计了。

恰当的原则应该是优先选择类而不是接口。从类开始，如果接口的必需性变得非常明确，那么就进行重构。接口是一种重要的工具，但是它们容易被滥用。

所选习题的答案都可以在名为The Thinking in Java Annotated Solution Guide的电子文档中找到，读者可以从www.MindView.net购买此文档。

343
~
344